Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 36

With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block. Design a C program to simulate the file allocation strategy.

Executable C code :

```
#include <stdio.h>

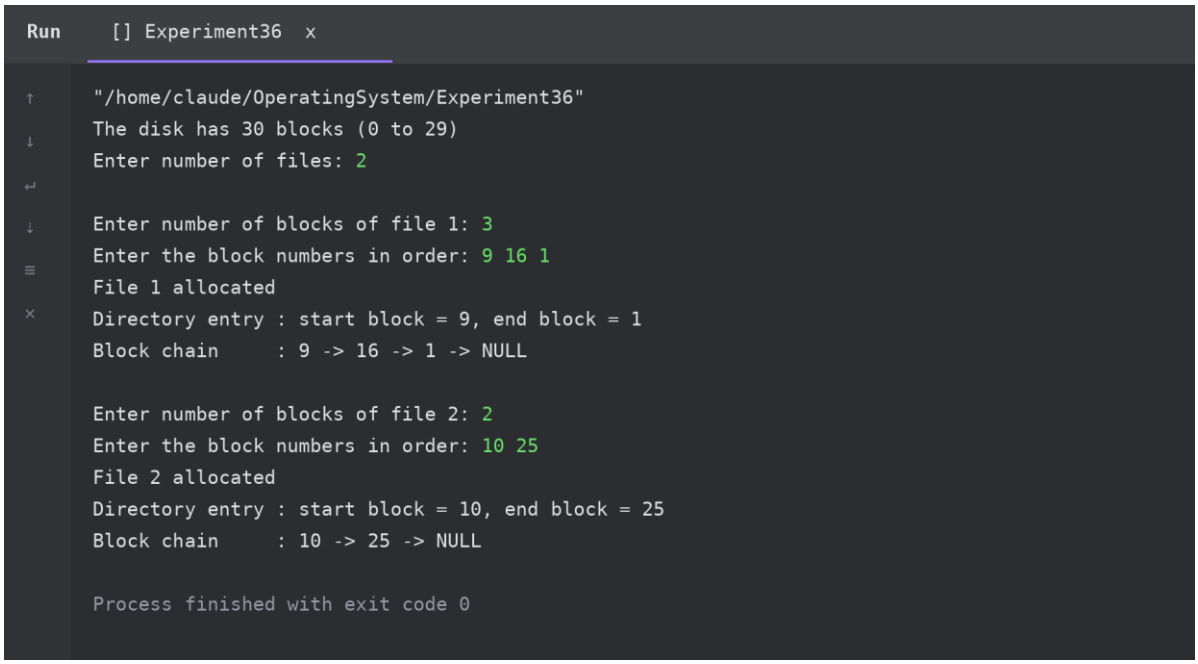
int main() {
    int used[30] = {0}, n, i, j, cnt, b[10], ok;

    printf("The disk has 30 blocks (0 to 29)\n");
    printf("Enter number of files: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("\nEnter number of blocks of file %d: ", i);
        scanf("%d", &cnt);
        printf("Enter the block numbers in order: ");
        for (j = 0; j < cnt; j++) scanf("%d", &b[j]);

        ok = 1;
        for (j = 0; j < cnt; j++) {
            if (b[j] < 0 || b[j] > 29 || used[b[j]]) {
                printf("File %d cannot be allocated: block %d is not free\n", i,
b[j]);
                ok = 0;
                break;
            }
        }
        if (ok) {
            for (j = 0; j < cnt; j++) used[b[j]] = 1;
            printf("File %d allocated\n", i);
            printf("Directory entry : start block = %d, end block = %d\n", b[0],
b[cnt - 1]);
            printf("Block chain     : ");
            for (j = 0; j < cnt; j++) printf("%d -> ", b[j]);
            printf("NULL\n");
        }
    }
    return 0;
}
```

Output Screenshot:



Experiment 37

Construct a C program to simulate the First Come First Served disk scheduling algorithm.

Executable C code :

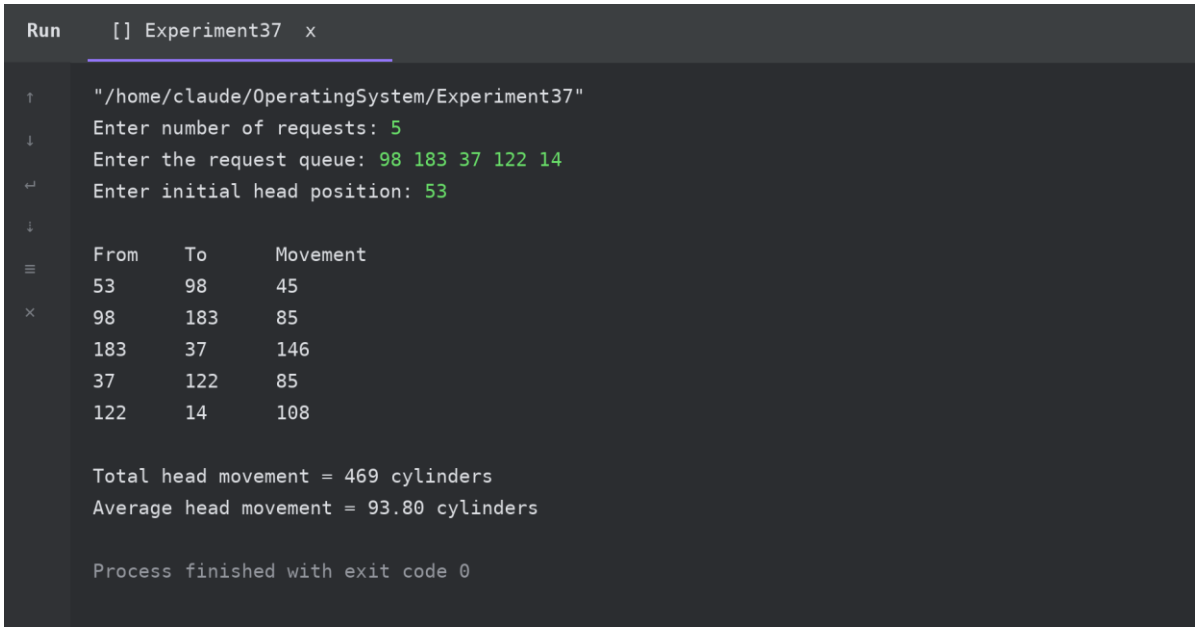
```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, req[20], head, cur, i, mv, total = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);

    cur = head;
    printf("\nFrom\tTo\tMovement\n");
    for (i = 0; i < n; i++) {                 // serve requests in the order they
arrived
        mv = abs(req[i] - cur);
        total += mv;
        printf("%d\t%d\t%d\n", cur, req[i], mv);
        cur = req[i];
    }
    printf("\nTotal head movement = %d cylinders\n", total);
    printf("Average head movement = %.2f cylinders\n", (float)total / n);
    return 0;
}
```

Output Screenshot:



Experiment 38

Design a C program to simulate SCAN disk scheduling algorithm.

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void move_to(int to) {
    int mv = abs(to - cur);
    total += mv;
    printf("%d\t%d\t%d\n", cur, to, mv);
    cur = to;
}

int main() {
    int n, req[20], head, size, dir, i, j, t, idx;

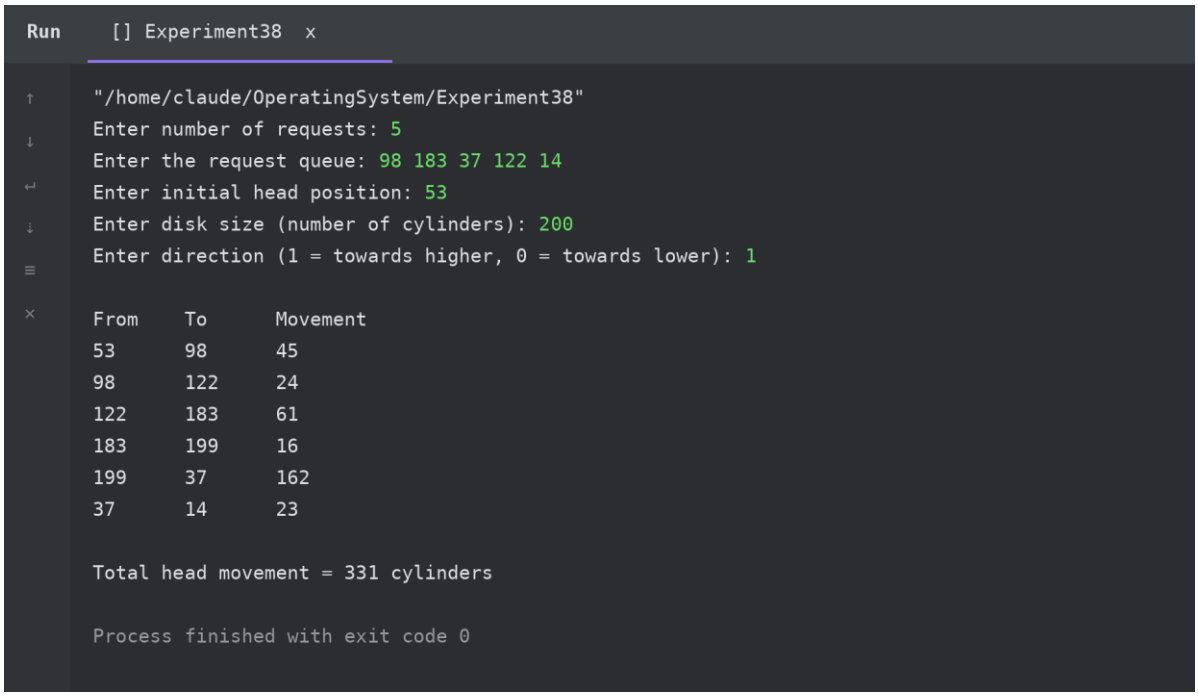
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size (number of cylinders): ");
    scanf("%d", &size);
    printf("Enter direction (1 = towards higher, 0 = towards lower): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)               // sort the requests
        for (j = i + 1; j < n; j++)
            if (req[i] > req[j]) { t = req[i]; req[i] = req[j]; req[j] = t; }

    for (idx = 0; idx < n && req[idx] < head; idx++);   // first request above
the head

    cur = head;
    printf("\nFrom\tTo\tMovement\n");
    if (dir == 1) {
        for (i = idx; i < n; i++) move_to(req[i]);       // go up serving
requests
        move_to(size - 1);                               // reach the last
cylinder
        for (i = idx - 1; i >= 0; i--) move_to(req[i]);  // reverse and serve the
rest
    } else {
        for (i = idx - 1; i >= 0; i--) move_to(req[i]);  // go down serving
requests
        move_to(0);                                      // reach cylinder 0
        for (i = idx; i < n; i++) move_to(req[i]);       // reverse and serve the
rest
    }
    printf("\nTotal head movement = %d cylinders\n", total);
    return 0;
}
```

Output Screenshot:



Experiment 39

Develop a C program to simulate C-SCAN disk scheduling algorithm.

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void move_to(int to) {
    int mv = abs(to - cur);
    total += mv;
    printf("%d\t%d\t%d\n", cur, to, mv);
    cur = to;
}

int main() {
    int n, req[20], head, size, i, j, t, idx;

    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size (number of cylinders): ");
    scanf("%d", &size);

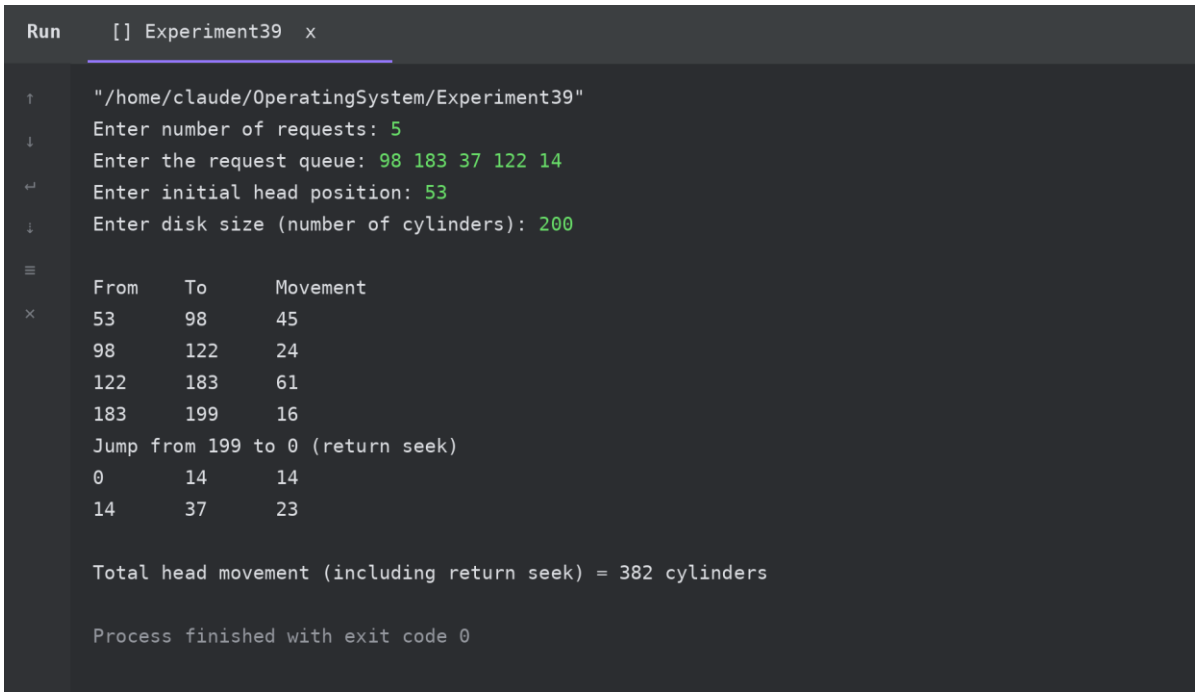
    for (i = 0; i < n - 1; i++)                // sort the requests
        for (j = i + 1; j < n; j++)
            if (req[i] > req[j]) { t = req[i]; req[i] = req[j]; req[j] = t; }

    for (idx = 0; idx < n && req[idx] < head; idx++);   // first request above
the head

    cur = head;
    printf("\nFrom\tTo\tMovement\n");
    for (i = idx; i < n; i++) move_to(req[i]);           // serve requests going
up
    move_to(size - 1);                                   // reach the last
cylinder
    printf("Jump from %d to 0 (return seek)\n", size - 1);
    total += size - 1;
    cur = 0;                                             // head returns to
cylinder 0
    for (i = 0; i < idx; i++) move_to(req[i]);           // serve the remaining
requests going up

    printf("\nTotal head movement (including return seek) = %d cylinders\n",
total);
    return 0;
}
```

Output Screenshot:



Experiment 40

Illustrate the various File Access Permission and different types users in Linux.

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <pwd.h>
#include <grp.h>
#include <sys/stat.h>

void show(const char *file) {
    struct stat st;
    char u[4] = "---", g[4] = "---", o[4] = "---", all[10];
    struct passwd *pw;
    struct group *gr;

    stat(file, &st);
    if (st.st_mode & S_IRUSR) u[0] = 'r';
    if (st.st_mode & S_IWUSR) u[1] = 'w';
    if (st.st_mode & S_IXUSR) u[2] = 'x';
    if (st.st_mode & S_IRGRP) g[0] = 'r';
    if (st.st_mode & S_IWGRP) g[1] = 'w';
    if (st.st_mode & S_IXGRP) g[2] = 'x';
    if (st.st_mode & S_IROTH) o[0] = 'r';
    if (st.st_mode & S_IWOTH) o[1] = 'w';
    if (st.st_mode & S_IXOTH) o[2] = 'x';
    sprintf(all, "%s%s%s", u, g, o);

    pw = getpwuid(st.st_uid);
    gr = getgrgid(st.st_gid);
    printf("\nFile        : %s\n", file);
    printf("Permissions : -%s (octal %o)\n", all, st.st_mode & 0777);
    printf("Owner       : %s\n", pw ? pw->pw_name : "unknown");
    printf("Group       : %s\n", gr ? gr->gr_name : "unknown");
    printf("\n%-26s%s\n", "User type", "Permission");
    printf("%-26s%s\n", "User   (owner)", u);
    printf("%-26s%s\n", "Group  (group members)", g);
    printf("%-26s%s\n", "Others (everyone else)", o);
}

int main() {
    char file[50];
    int mode;

    printf("Enter a file name to create: ");
    scanf("%s", file);
    printf("Enter permission in octal (for example 754): ");
    scanf("%o", &mode);

    close(open(file, O_CREAT | O_WRONLY, 0644));
    chmod(file, mode);                    // set the permissions given by the
user
    show(file);

    printf("\nr = read (4), w = write (2), x = execute (1)\n");
    printf("Each digit of the octal number is the sum for User, Group and
Others\n");
    unlink(file);
    return 0;
}
```

Output Screenshot:

